

Abstract and Conception Documentation

StudyFlow: An Academic Workload Planning System for University Students

Course: Project: Software Engineering (DLMCSPSE01)

1. Abstract

Problem Statement. University students are often required to manage lectures, assignments, coursework, revision, and submission deadlines at the same time. Although university portals provide official information such as course materials, schedules, and announcements, they usually do not help students organize their workload in a personalized and practical way. Students may know that a deadline exists, but still struggle to decide which task to begin first, how much time to dedicate to each assignment, and how to balance academic work with a busy class timetable. This problem becomes more significant when several assignments are due close together or when students already have heavily occupied lecture schedules. In such cases, poor planning can lead to missed deadlines, rushed submissions, increased stress, and inefficient study habits.

Proposed Solution. StudyFlow is a browser-based academic workload planning system designed to help university students manage assignments, schedules, and study planning within one platform. The system combines assignment and subject management, class schedule import through .ics files, recommendation generation, a planner, and a weekly calendar view. Students can create and manage subjects and assignments, upload lecture schedules, import subjects from saved calendar entries, receive priority-based study recommendations, and view suggested study sessions alongside class events. This makes the platform more useful than a basic to-do list because it connects workload planning with timetable awareness.

Expected Impact. The expected impact of this project is improved academic organization and more realistic workload planning for students. By combining assignments with class schedule information, the system helps students distribute effort more effectively, make better use of available time, and reduce last-minute work. This can support improved time management, reduced academic stress, and more consistent study habits. More broadly, the system encourages students to think about academic planning in relation to actual schedule constraints rather than deadlines alone.

2. Project Profile and Methodology

2.1 Objectives and Scope

The objective of StudyFlow is to provide a web-based application that helps university students organize and plan academic work in a more realistic way. The scope of the project includes user authentication, subject management, assignment tracking, calendar import from .ics files, recommendations based on assignment urgency and schedule intensity, a planner for study sessions, and a weekly calendar view that combines class events with generated study sessions. The project focuses on the core use case of helping students understand what they need to work on, when they should work on it, and how that work fits around their lecture schedule.

2.2 Target Group

The target group consists primarily of university students who handle multiple deadlines and class commitments at the same time. These users usually need support in organizing assignments, balancing study time with lectures, and avoiding last-minute work. A secondary benefit is that the

system can also be useful for students who prefer a more structured and visual planning process than generic reminder or notes applications provide.

2.3 Risks Associated with the Project

- Feature scope growth as additional planning and calendar features are introduced during the project lifecycle.
- Complexity in planner logic, especially when multiple assignments share close deadlines and different workload estimates.
- Variation in calendar data quality, because uploaded .ics files may contain inconsistent titles, formats, or time structures.
- Deployment and persistence risks related to backend hosting, file uploads, sessions, and data storage in a browser-based application.
- Usability risk that a planning feature may become too complex if too many controls or views are introduced without clear navigation.

2.4 Project Plan

Stage	Main Focus
Iteration 1	Authentication, subjects, assignments, and dashboard summaries
Iteration 2	Calendar upload, saved class events, and subject import from calendar
Iteration 3	Recommendations, planner logic, study sessions, and weekly calendar view
Iteration 4	Interaction improvements, UI polish, documentation, and deployment preparation

2.5 Project Organization

The project organization is intentionally lightweight because the system was designed and implemented within the scope of a course project. Roles such as requirements analysis, design, implementation, testing, and documentation are handled in a compact form, with strong emphasis on traceability between requirements, design decisions, and implemented functionality. This makes prioritization and scope control especially important throughout the project.

2.6 Chosen Software Development Methodology

The project follows an iterative, agile-inspired development methodology. This choice was made because the system evolved through several connected features that benefited from progressive refinement rather than a single sequential implementation. Core features such as authentication, subject management, and assignments were implemented first. More advanced features such as calendar import, subject extraction from calendar entries, recommendations, planner logic, and interactive calendar visualization were added afterwards. This methodology fits the project well because it allowed the application to remain usable after each iteration, reduced implementation risk, and made it easier to adapt the design when better ideas emerged during development.

3. Requirements and Glossary

3.1 Functional Requirements

- **FR1:** The system shall allow users to sign up, log in, and log out securely.
- **FR2:** The system shall allow users to create, edit, delete, and view subjects.

- **FR3:** The system shall allow users to create, edit, delete, view, and filter assignments.
- **FR4:** The system shall allow users to upload an .ics timetable and store class events.
- **FR5:** The system shall allow users to import candidate subjects from saved calendar entries.
- **FR6:** The system shall generate recommendations based on class schedule, due dates, estimated hours, and priorities.
- **FR7:** The system shall generate suggested study sessions and display them in planner and calendar views.
- **FR8:** The system shall display class events and study sessions in a weekly calendar view.
- **FR9:** The system shall allow users to move and resize study sessions in the calendar view while keeping class events fixed.

3.2 Non-Functional Requirements

- **NFR1 - Usability:** The application should be self-explanatory and usable through a browser without special technical knowledge.
- **NFR2 - Performance:** Normal operations such as loading assignments, viewing schedules, and generating planner sessions should respond quickly for a student-sized dataset.
- **NFR3 - Maintainability:** The project should remain modular enough that features such as recommendations, planner logic, and calendar handling can be improved independently.
- **NFR4 - Reliability:** Uploaded calendar data and generated study sessions should be stored consistently in the database.
- **NFR5 - Traceability:** Source code and project artifacts should be maintained in GitHub with meaningful commit history and documentation.

3.3 Glossary

Term	Meaning in StudyFlow
Subject	An academic module or course such as Algorithmics or Computer Science and Society.
Assignment	A task linked to a subject with a due date, priority, status, and estimated study effort.
Class Event	A lecture or timetable event imported from an .ics calendar file.
Study Session	A suggested or manually adjusted study block generated for assignment work.
Recommendation	A rule-based planning suggestion derived from workload, deadlines, and class schedule intensity.
Planner	A view that transforms assignments and schedule constraints into suggested study allocation across days.

4. Initial System Design

4.1 System Scope and Context

The system context is centered around one main actor: the student user. The user accesses the browser-based application, manages subjects and assignments, uploads a timetable in .ics format, reviews generated recommendations, uses the planner, and sees both class events and study sessions in a weekly calendar view. The application depends on uploaded calendar data as an external input and uses a relational database for persistence. Within the defined project scope, the system is designed as a self-contained web application without dependency on external university APIs.

4.2 Building Block View

- **Presentation Layer:** HTML templates and browser-based UI for dashboard, assignments, subjects, calendar, recommendations, planner, and calendar view.
- **Application Logic Layer:** Route handlers and planning logic for authentication, assignments, calendar processing, recommendations, planner generation, and study session updates.
- **Persistence Layer:** Database-backed models for users, subjects, assignments, class events, and study sessions.
- **Import/Transformation Layer:** .ics parsing and conversion of class schedule data into stored class events and subject candidates.

4.3 Technologies and Tools

Technology / Tool	Reason for Selection
FastAPI	Lightweight Python web framework suitable for backend routes, forms, session-based flows, and quick development.
Jinja2 Templates	Simple server-rendered UI approach suitable for a student project without heavy frontend framework overhead.
SQLAlchemy	Provides structured ORM support for users, assignments, class events, and study sessions.
SQLite	Simple local database option for development and demonstration without deployment complexity.
icalendar	Used to parse .ics timetable files and extract class events.
FullCalendar	Used to provide a proper weekly calendar interface instead of a plain list-based pseudo-calendar.
GitHub	Used for version control, source code hosting, and portfolio requirements.

4.4 Design Motivation

The chosen design aims to balance implementation scope, clarity, and customer value. A browser-based server-rendered application was selected because it satisfies the requirement of an intuitive web application while remaining manageable within the available scope. The design also keeps features logically separated: calendar import handles schedule data, recommendations reason about urgency and workload, planner logic generates study sessions, and the calendar view presents a more intuitive visual schedule. This separation supports maintainability and makes future improvements possible.

4.5 Testing Consideration at Conception Stage

From the conception stage, testing is considered in relation to both functional and non-functional requirements. Representative test scenarios include account creation and login, subject and assignment CRUD operations, import of valid and invalid .ics files, generation of recommendations for urgent and non-urgent assignments, creation of study sessions, and movement of generated study sessions in the calendar view. These tests help verify that the implemented features solve the basic problem of academic workload planning in a practical and usable way.